

TRD.md

Technical Requirements Document

Project: Bloom

Document: Technical Requirements Document (TRD)

Version: 1.0

Status: Engineering Specification

1. Purpose

This document defines the technical requirements for Bloom.

Unlike the PRD, which defines **what** the product should do, the TRD defines **how the system should behave from an engineering perspective**.

It serves as the primary reference for software engineers, architects, DevOps engineers, QA engineers, and AI engineers.

2. Scope

This document covers:

- Functional requirements
- Non-functional requirements
- System architecture constraints
- Scalability
- Performance
- Security
- Reliability
- Observability
- AI system requirements
- Integration requirements
- Deployment requirements

It intentionally excludes UI design and business strategy.

3. System Overview

Bloom is a distributed AI-powered Learning Intelligence Platform composed of multiple independent services.

Core responsibilities include:

- User management
- Learning profile management
- AI recommendation
- Knowledge graph reasoning
- Content discovery
- Progress tracking
- Analytics
- Search
- Notifications

The system should be modular, scalable, and cloud-native.

4. Functional Requirements

4.1 User Management

The system shall:

- Support secure user registration.
 - Support authentication.
 - Support OAuth providers.
 - Support password recovery.
 - Support profile management.
 - Support account deletion.
 - Support multi-device sessions.
-

4.2 Learning Profile

The system shall:

- Store career goals.
- Store learning preferences.
- Track completed resources.
- Maintain learning history.
- Estimate learner knowledge.

- Update learner profiles continuously.
-

4.3 Recommendation Engine

The system shall:

- Generate personalized recommendations.
 - Rank educational resources.
 - Support explainable recommendations.
 - Adapt recommendations using learner feedback.
 - Refresh recommendations dynamically.
-

4.4 Skill Graph

The system shall:

- Represent skill dependencies.
 - Model prerequisite relationships.
 - Support graph traversal.
 - Support career-to-skill mapping.
 - Support skill gap analysis.
-

4.5 Resource Management

The system shall:

- Index external learning resources.
 - Maintain metadata.
 - Remove outdated resources.
 - Deduplicate similar resources.
 - Categorize educational content.
-

4.6 Progress Tracking

The system shall:

- Track learning sessions.
- Measure progress.
- Estimate mastery.
- Generate learning statistics.
- Support milestones.

4.7 Search

The system shall provide:

- Keyword search
- Semantic search
- Skill search
- Creator search
- Career search

4.8 Notifications

The system shall:

- Send reminder notifications.
- Recommend revision sessions.
- Notify learning milestones.
- Notify roadmap updates.

5. Non-Functional Requirements

Availability

Target uptime:

99.9%

Reliability

The platform should recover gracefully from failures.

No single service should become a single point of failure.

Scalability

The architecture shall support horizontal scaling.

Application services should remain stateless whenever possible.

Performance

Target API response time:

- Read operations: <200 ms (excluding external API latency)
- Recommendation retrieval: <500 ms for cached recommendations
- Search results: <300 ms

Security

All sensitive data must be encrypted.

Authentication tokens must expire.

Passwords must never be stored in plain text.

Communication must occur over HTTPS.

Maintainability

Services should:

- Have clear boundaries.
- Be independently deployable.
- Follow consistent coding standards.
- Support automated testing.

Portability

Bloom should be deployable on multiple cloud providers.

Vendor-specific dependencies should be minimized.

6. System Architecture Constraints

The architecture should follow:

- Modular design

- Service-oriented architecture
 - Event-driven communication where appropriate
 - API-first development
 - Stateless application services
 - Independent scaling of services
-

7. AI Requirements

The AI system shall:

- Build learner profiles.
- Estimate skill mastery.
- Generate recommendations.
- Explain recommendations.
- Learn from feedback.
- Support continuous model improvements.

The AI must never make irreversible decisions without user control.

8. Data Requirements

The platform shall store:

- User profiles
- Learning history
- Skill graph
- Career graph
- Creator metadata
- Content metadata
- Recommendation history
- Analytics events
- Feedback events

Data should be versioned where appropriate.

9. API Requirements

All APIs should:

- Be versioned.
- Return structured error responses.

- Use consistent naming conventions.
- Support pagination where required.
- Validate inputs.
- Provide appropriate HTTP status codes.

Authentication is required for protected endpoints.

10. Integration Requirements

Bloom should integrate with:

- Authentication providers
- Learning content platforms
- Analytics services
- Notification providers
- Payment providers (future)
- Enterprise identity providers (future)

Integrations should be abstracted behind service interfaces to avoid tight coupling.

11. Security Requirements

Authentication

- Secure login
- Token-based authentication
- Session management

Authorization

- Role-based access control
- Least-privilege access

Data Protection

- Encryption in transit
- Encryption at rest
- Secure secret management

Audit

The platform should log:

- Authentication events

- Administrative actions
 - Critical system changes
-

12. Privacy Requirements

Bloom should follow privacy-by-design principles.

Requirements include:

- Minimal data collection
- User consent where applicable
- Data export capability
- Account deletion capability
- Configurable data retention policies

The learner remains in control of their personal data.

13. Observability

Every service should expose:

Logs

- Structured logs
- Error logs
- Audit logs

Metrics

- API latency
- Error rate
- CPU usage
- Memory usage
- Recommendation latency
- Queue length

Tracing

- Distributed request tracing
 - Service dependency visualization
-

14. Monitoring

The engineering team should monitor:

Infrastructure

- CPU
- Memory
- Disk
- Network

Application

- API failures
- Authentication failures
- Search latency
- Recommendation latency

Business

- Active users
 - Recommendation acceptance
 - Learning session completion
-

15. Deployment Requirements

The platform should support:

- Continuous Integration
- Continuous Delivery
- Rolling deployments
- Blue-green deployments
- Automatic rollback
- Environment isolation

Deployment environments:

- Development
 - Testing
 - Staging
 - Production
-

16. Testing Requirements

Testing strategy should include:

Unit Testing

- Business logic
- Utilities
- AI components (where deterministic)

Integration Testing

- API communication
- Database interactions
- Third-party integrations

End-to-End Testing

- Complete learner journeys
- Recommendation workflows
- Authentication flows

Performance Testing

- Load testing
- Stress testing
- Scalability testing

Security Testing

- Vulnerability scanning
 - Authentication testing
 - Authorization testing
-

17. Failure Handling

The system should:

- Retry transient failures.
- Use circuit breakers for unstable dependencies.
- Return meaningful error messages.
- Log unexpected exceptions.
- Avoid cascading failures.

Critical user data should not be lost during recoverable failures.

18. Technical Risks

Potential risks include:

- Recommendation quality degradation
- External platform dependency
- AI model drift
- Rapid data growth
- Search performance degradation
- Graph complexity growth

Mitigation strategies should be documented and reviewed regularly.

19. Engineering Standards

The engineering team should follow:

- Version control using Git
 - Code reviews before merging
 - Automated testing in CI
 - Semantic versioning
 - API documentation
 - Infrastructure as Code where practical
 - Consistent coding standards
-

20. Success Criteria

The technical implementation is considered successful when it:

- Meets defined performance targets.
 - Achieves availability objectives.
 - Produces accurate and explainable recommendations.
 - Scales to increasing numbers of users without architectural redesign.
 - Protects user data through secure engineering practices.
 - Supports rapid iteration through modular and maintainable services.
-

21. Future Technical Considerations

Future versions may introduce:

- Real-time recommendation updates
- Federated learning
- Offline-first mobile support
- Multi-region deployments
- Edge inference
- Advanced graph reasoning
- Personalized AI agents
- Plugin ecosystem

These capabilities should build upon the same modular architecture without requiring fundamental redesign.

TRD Summary

The Technical Requirements Document defines the engineering foundation of Bloom. It establishes the functional capabilities, quality attributes, operational requirements, and architectural constraints needed to build a scalable, secure, maintainable, and AI-driven Learning Intelligence Platform.

The guiding principle is:

Every technical decision should improve reliability, scalability, security, and the learner's experience while preserving flexibility for future innovation.